RESEARCH ARTICLE

A Comparative Study of Process Synchronization Algorithms

Monitor · Semaphore · Peterson · Dekker · Readers–Writers

ARTICLE INFORMATION

Field	Details
Title	Comparison of Monitor, Semaphore, Peterson, Dekker and Readers–Writers
Author	Zeynab Rezaii
Course	Embedding Systems / Concurrent Programming
Supervisor	Majid Shamshirgaran
University / Department	Eshragh Univesity
Language	English
Date	November ۲۰۲۶
Keywords	synchronization, critical section, mutual exclusion, semaphore, monitor

Contents: pseudocode · flowcharts · charts · comparison tables · simulation of Readers–Writers policies

Table of Contents

١. Introduction and Problem Definition ٣
٢. Evaluation Criteria ٤
٣. Dekker's Algorithm ٤
٤. Peterson's Algorithm ٦
٥. Semaphores ٨
٦. Monitors ٩
٧. The Readers–Writers Problem ١١
٨. Comprehensive Comparison Tables ١٦
٩. Comparison Charts ١٩
١٠. Algorithm Selection Guide ٢١
١١. Conclusion ٢٢
١٢. References ٢٢

> Note: figures, flowcharts and pseudocode use English identifiers.

Abstract

Synchronization of concurrent processes is one of the most fundamental problems in operating systems and parallel programming. This article studies and compares five classic solutions: **Dekker's algorithm**, **Peterson's algorithm**, **semaphores**, **monitors** and the **Readers–Writers** problem. For each approach we give pseudocode, a flowchart, an analysis of the three correctness requirements (mutual exclusion, progress and bounded waiting) and evaluation charts. Comprehensive comparison tables cover parameters such as waiting type, starvation, deadlock, scalability, hardware requirements and efficiency. A discrete-event simulation additionally measures the behaviour of three Readers–Writers policies under different loads. Finally, a guide for choosing a suitable algorithm is provided.

Keywords: synchronization, critical section, mutual exclusion, busy waiting, semaphore, monitor, Peterson, Dekker, readers–writers.

۱ Introduction and Problem Definition

۱,۱ The critical-section problem

When several processes (or threads) access shared data concurrently, the result depends on the exact interleaving of their instructions. This situation is called a **race condition**. The part of a program that touches the shared resource is the **critical section**. Every process is modelled as four parts: an entry section, the critical section, an exit section and a remainder section.

Pseudocode ۱ — General structure of a process

```
do {
    entry section        // request permission to enter
    critical section     // access the shared resource
    exit section         // announce that we have left
    remainder section    // private work
} while (true);
```

۱,۲ Three correctness requirements

A correct solution must satisfy three requirements simultaneously. **Mutual exclusion**: at most one process is inside the critical section at any time. **Progress**: if the critical section is free and some processes want to enter, the decision about who enters cannot be postponed indefinitely, and only the requesting processes take part in it. **Bounded waiting**: between a request to enter and its grant, the number of times other processes enter first is bounded, so no starvation occurs. The standard assumptions are: the relative speeds of processes are unknown, a process stays in the critical section for a finite time, and reading or writing a single variable is atomic.

۱,۳ Roadmap and historical context

The five approaches fall into three levels of abstraction. **Dekker** and **Peterson** are pure-software solutions for two processes that use busy waiting. A **semaphore** is a primitive supported by the operating system or hardware, and a **monitor** is a language-level construct that provides mutual exclusion automatically. **Readers–Writers** is a classic coordination problem solved with semaphores or monitors. Figure ۱ shows when these approaches appeared.

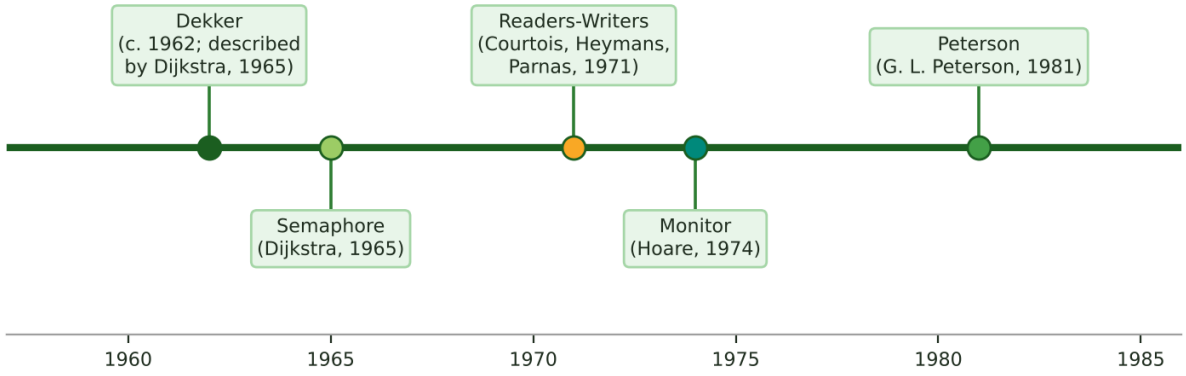


Figure ١: *Chronology of the synchronization approaches*

٢ Evaluation Criteria

For a fair comparison all approaches are evaluated against the same fixed set of parameters. Table ١ defines each parameter in both languages; the same definitions are used in the following sections.

Table ١: Evaluation parameters

Parameter	Definition
Mutual exclusion	At most one process in the critical section at a time.
Progress	The decision to enter is never postponed forever.
Bounded waiting	Upper bound on how often others enter first.
Starvation	A process that never gets its turn.
Deadlock	Processes waiting for each other forever.
Waiting type	Busy waiting (spinning on the CPU) or blocking (sleeping).
Scalability	Behaviour as the number of processes grows.
Hardware / OS support	Which atomic or OS support is required.
Memory overhead	Number of shared variables and queues needed.
Complexity and proof	Difficulty of implementation and correctness proof.
CPU efficiency	Whether processor time is wasted while waiting.

Besides the qualitative parameters above, two countable structural indicators are extracted from the pseudocode of this article: the number of shared synchronization variables and the number of entry/exit statements. For Readers–Writers, performance indicators (average and maximum waiting time) come from a simulation. The radar scores in Section ٩ are the author's qualitative assessment based on the analysis, not empirical measurements.

٣ Dekker's Algorithm

٣,١ Overview

Dekker's algorithm is the first correct software solution to the mutual-exclusion problem for two processes. It was devised by the Dutch mathematician Th. J. Dekker around ١٩٦٢ and described by Dijkstra in ١٩٦٥ in “Cooperating Sequential Processes”. It needs only atomic reads and writes of shared variables; no special hardware instruction such as Test-and-Set is required.

٣,٢ Idea of the algorithm

Each process Pi announces its intention to enter by raising its flag `flag[i]`. If the rival's flag is also raised, there is contention and the variable `turn` decides whose turn it is. The process that does not hold the turn lowers its flag temporarily, waits until it is its turn, and then raises the flag again. On exit, the process hands the turn to the rival and lowers its flag. This “polite back-off” prevents deadlock; a naive version using only `flag` can deadlock, and a version using only `turn` forces the processes to alternate strictly.

٣,٣ Pseudocode

Pseudocode ٢ — Dekker's algorithm (process Pi, j = ١ - i)

```
shared: boolean flag[2] = {false, false};   int turn = 0;

do {
    flag[i] = true;                 // I want to enter
    while (flag[j]) {               // rival also wants to enter
        if (turn == j) {            // rival has priority
            flag[i] = false;        // back off
            while (turn == j) ;     // busy wait for my turn
            flag[i] = true;         // try again
        }
    }
    /* ------- CRITICAL SECTION ------- */
    turn = j;                       // give priority to rival
    flag[i] = false;                // I am done
    /* ------- REMAINDER SECTION ------ */
} while (true);
```

٣,٤ Flowchart

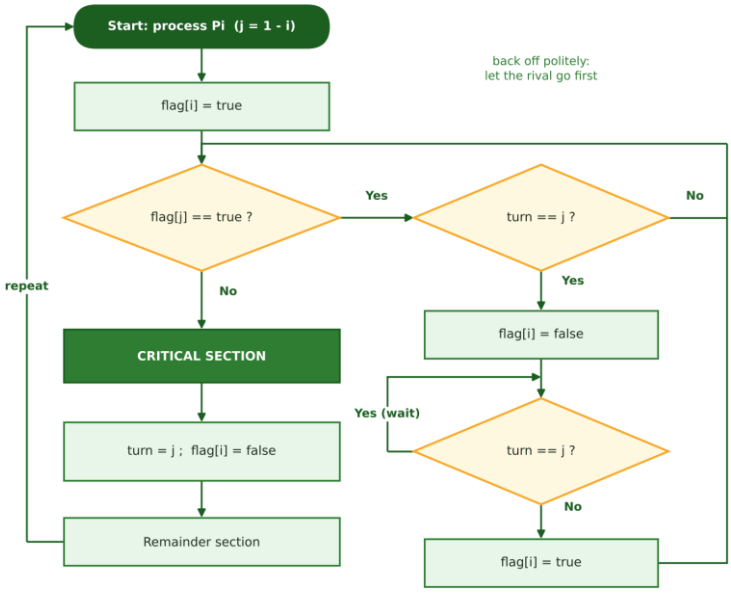


Figure ٢: *Flowchart of Dekker's algorithm for process Pi*

٣,٥ Analysis

Mutual exclusion: a process that sees `flag[j]` lowered already has its own flag raised, so the rival either stays in its wait loop or backs off after seeing that flag. **Progress:** `turn` always gives priority to exactly one of the two processes, and under contention the holder proceeds without waiting. **Bounded waiting:** on exit the turn is handed to the rival, so each process lets the rival enter ahead of it at most once.

- **Strengths:** fully correct for two processes; no special atomic instruction; all three requirements hold.
- **Weaknesses:** busy waiting wastes CPU; intricate logic and hard proof; only two processes (Dijkstra's N-process generalization needs ٢N+١ variables); fails on modern processors without memory barriers.

٤ Peterson's Algorithm

٤,١ Overview and idea

In ١٩٨١ Gary L. Peterson published a much simpler solution than Dekker's. Each process first announces its need with `flag[i] = true` and then "gives the turn away" to its rival (`turn = j`). It waits only if the rival also wants to enter and the turn belongs to the rival. Because both processes write `turn` and the last writer loses, only one of them can pass the test.

٤,٢ Pseudocode

Pseudocode ٣ — Peterson's algorithm (process Pi, j = ١ - i)

```
shared: boolean flag[2] = {false, false};   int turn;

do {
    flag[i] = true;                          // I am interested
    turn = j;                                // let the rival go first
    while (flag[j] && turn == j) ;           // busy wait
    /* ------- CRITICAL SECTION ------- */
    flag[i] = false;                         // exit section
    /* ------- REMAINDER SECTION ------ */
} while (true);
```

Pseudocode ٤ — Generalization to N processes (filter lock)

```
shared: int level[N] = {-1,...,-1};   int victim[N];

lock(i):
    for (L = 0; L < N - 1; L++) {            // N-1 filtering levels
        level[i] = L;                        // I reached level L
        victim[L] = i;                       // I am the last arrival
        while (exists k != i : level[k] >= L && victim[L] == i) ;
    }
    /* ------- CRITICAL SECTION ------- */
unlock(i):
    level[i] = -1;
```

٤,٣ Flowchart and execution trace

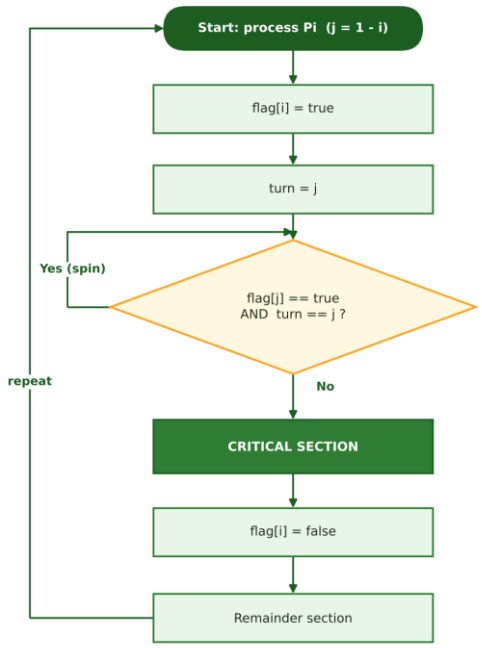


Figure ٣: *Flowchart of Peterson's algorithm for process Pi*

Figure ٤ shows one possible interleaving of the two processes. Both raise their flags and write `turn`; since P١ is the last writer of `turn` (final value ٠), P٠ passes the test and enters, while P١ busy-waits until P٠ leaves.

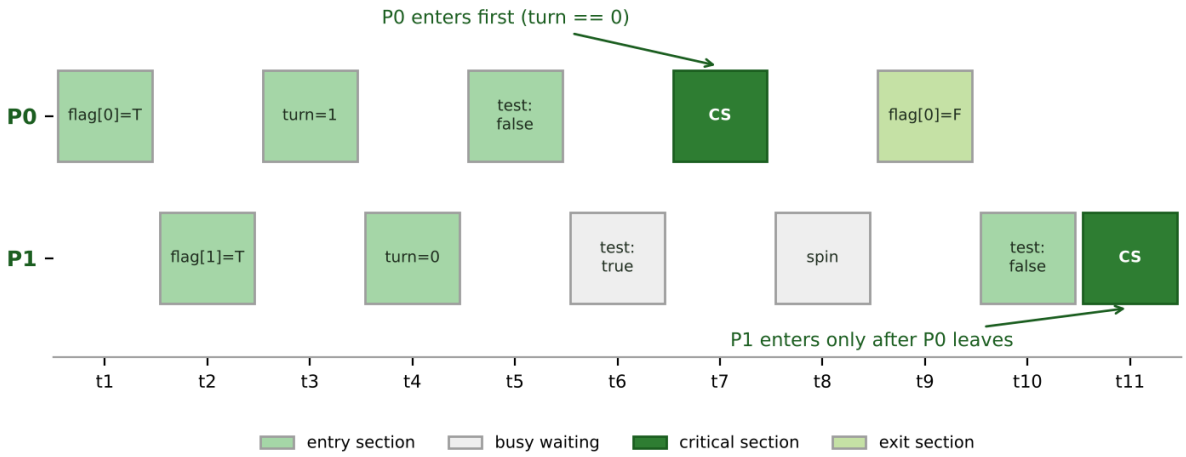


Figure ٤: *Timing trace of one possible execution of Peterson's algorithm*

٤,٤ Correctness argument and analysis

Short proof of mutual exclusion: assume P٠ passes the entry test before P١. At that moment either `flag[1]` was lowered (then P١ later writes `turn = 0`, and because P٠ does not change `turn` until it exits and `flag[0]` is raised, P١ stays blocked), or `turn = 0` already held (P١ had written it earlier, and that same state keeps P١ blocked). In both cases P١ cannot enter simultaneously. **Progress and bounded waiting:** the process that wrote `turn` last waits, and the rival lowers its `flag` on exit, so each process enters at most once after the rival.

- **Strengths:** very short and provable; all three requirements hold; generalizes to N processes via the filter lock.

- **Weaknesses:** busy waiting; the filter lock needs O(N) entry cost and ٢N-١ memory cells; on modern hardware it fails without memory fences or sequentially consistent atomics because stores may be reordered after later loads (store buffers).

٥ Semaphores

٥,١ Overview

Semaphores were introduced by Dijkstra in ١٩٦٥. A semaphore is an integer variable `S` that can be changed only through two atomic operations, `wait(S)` (also P) and `signal(S)` (also V). A **binary semaphore** (mutex) holds only ٠ or ١ and is used for mutual exclusion; a **counting semaphore** can hold any non-negative value and records the number of free instances of a resource. Semaphores have three main uses: mutual exclusion (initial value ١), event ordering (initial value ٠) and resource counting (initial value equal to the capacity).

٥,٢ Pseudocode

Pseudocode ٥a — Busy-waiting semaphore

```
wait(S):   while (S <= 0) ;   // spin until positive
           S--;
signal(S): S++;
```

Pseudocode ٥b — Blocking semaphore (no busy waiting)

```
typedef struct { int value; queue list; } semaphore;

wait(S):   S.value--;
           if (S.value < 0) { add this process to S.list; block(); }
signal(S): S.value++;
           if (S.value <= 0) { remove a process P from S.list; wakeup(P); }
```

Pseudocode ٦ — Mutual exclusion with a binary semaphore

```
semaphore mutex = 1;

do {
    wait(mutex);          // entry section
    /* ------- CRITICAL SECTION ------- */
    signal(mutex);        // exit section
    /* ------- REMAINDER SECTION ------ */
} while (true);
```

Pseudocode ٧ — Bounded buffer (producer-consumer) with three semaphores

```
semaphore mutex = 1, empty = N, full = 0;

Producer:                          Consumer:
  do {                               do {
    produce(item);                     wait(full);
    wait(empty);                       wait(mutex);
    wait(mutex);                       item = remove();
    insert(item);                      signal(mutex);
    signal(mutex);                     signal(empty);
    signal(full);                      consume(item);
  } while (true);                    } while (true);
```

٥,٣ Flowchart

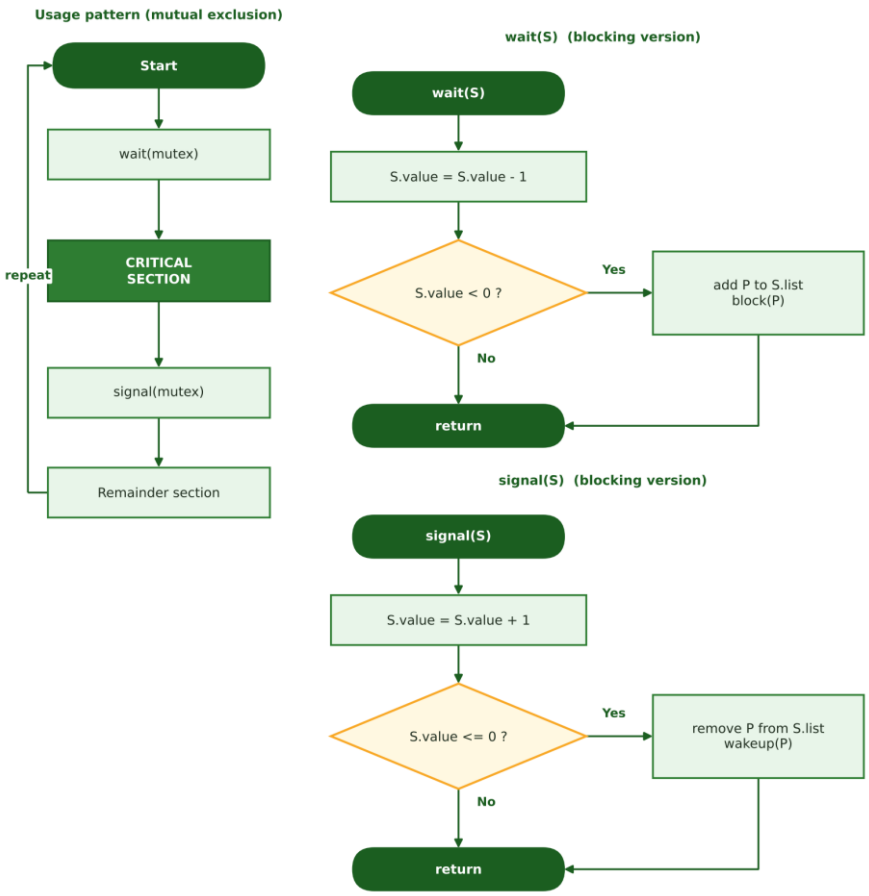


Figure ٥: *Usage pattern of a semaphore and the blocking implementation of wait and signal*

٥,٤ Analysis

Correctness: because `wait` and `signal` are atomic (implemented with interrupt disabling, a spinlock or a Test-and-Set instruction) and the semaphore value cannot be decremented simultaneously by two processes, mutual exclusion holds. **Bounded waiting** depends on the queue policy: it holds for a **strong** semaphore (FIFO queue) but starvation is possible with a **weak** semaphore (arbitrary wake-up). In the blocking version a waiting process consumes no CPU.

- **Hazards:** deadlock from a wrong ordering (e.g. swapping `wait(empty)` and `wait(mutex)` in the producer), forgetting `signal`, and **priority inversion**, famously seen in the Mars Pathfinder mission in ١٩٩٧.
- **Strengths:** scales to N processes, no busy waiting (blocking version), flexible for ordering and resource counting. **Main weakness:** low level and error-prone, since mutual exclusion relies on programmer discipline rather than language structure.

٦ Monitors

٦,١ Overview

Monitors were proposed by C. A. R. Hoare in ١٩٧٤ (and independently by Per Brinch Hansen) to overcome the problems of semaphores. A monitor is a language-level construct that encapsulates shared data together with the procedures that access it and **allows only one process to be active inside it at a time**; mutual exclusion is guaranteed by the compiler or runtime. Waiting for a condition uses **condition variables** with the operations

`wait()` and `signal()`. `c.wait()` releases the monitor lock and puts the process to sleep on the condition queue; `c.signal()` wakes up one waiting process.

٦٫٢ Signalling semantics

There are two main styles. In the **Hoare** style (signal-and-wait) the signalling process hands the monitor to the woken process immediately, so the condition is guaranteed to hold when it resumes and an `if` is sufficient. In the **Mesa** style (signal-and-continue) the signaller keeps running and the woken process must re-acquire the lock; since the condition may have changed in between, the test must be repeated inside a `while` loop. Java, pthreads and most modern languages follow the Mesa style.

٦٫٣ Pseudocode

Pseudocode ٨ — Bounded buffer implemented as a monitor (Mesa semantics)

```
monitor BoundedBuffer {
    item buffer[N];  int count = 0, in = 0, out = 0;
    condition notFull, notEmpty;

    procedure put(item x) {
        while (count == N) notFull.wait();     // wait until space exists
        buffer[in] = x;  in = (in + 1) % N;  count++;
        notEmpty.signal();                     // wake one consumer
    }
    procedure item get() {
        while (count == 0) notEmpty.wait();    // wait until data exists
        x = buffer[out];  out = (out + 1) % N;  count--;
        notFull.signal();                      // wake one producer
        return x;
    }
}
```

٦,٤ Flowchart

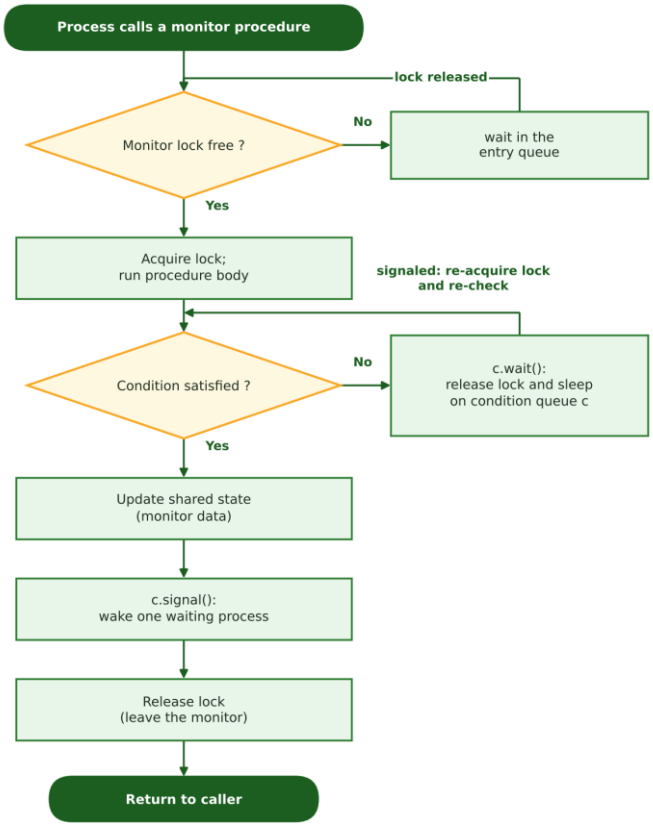


Figure ٦: *Flowchart of entering a monitor, waiting on a condition variable and leaving*

٦,٥ Analysis

Correctness: mutual exclusion is inherent and automatic, and the programmer cannot forget the entry or exit operations. **Progress:** because `wait()` releases the lock, other processes can proceed. **Bounded waiting** depends on the policy of the entry and condition queues (FIFO queues guarantee it). Waiting is blocking and consumes no CPU. Typical problems: **nested-monitor lockout**, lost wake-ups when `if` is used instead of `while` under Mesa semantics, and dependence on language support.

- **Strengths:** high level, safer than semaphores, data encapsulation, scalable, no busy waiting.
  Implementations: `synchronized/wait/notify` in Java, `mutex + cond_var` in pthreads, `Monitor` in C#.

٧ The Readers–Writers Problem

٧,١ Problem statement

The Readers–Writers problem was posed by Courtois, Heymans and Parnas in ١٩٧١. A shared resource (for example a database or a file) can be read by several **readers** at the same time, but a **writer** needs exclusive access. The rules: writers must not overlap with writers or readers, while readers do not conflict with each other. Three variants are defined by priority: **reader priority** (first problem), **writer priority** (second problem) and **starvation-free / fair** (third problem).

٧,٢ Pseudocode of the three solutions

Pseudocode ٩ — First problem: reader priority (semaphores)

```
semaphore mutex = 1, wrt = 1;   int readcount = 0;

Reader:                                 Writer:
  wait(mutex);                            wait(wrt);
  readcount++;                            /* WRITING */
  if (readcount == 1) wait(wrt);          signal(wrt);
  signal(mutex);
  /* READING */
  wait(mutex);
  readcount--;
  if (readcount == 0) signal(wrt);
  signal(mutex);
```

Pseudocode ١٠ — Second problem: writer priority (semaphores)

```
semaphore rmutex = 1, wmutex = 1, wrt = 1, readTry = 1;
int readcount = 0, writecount = 0;

Reader:                                 Writer:
  wait(readTry);                          wait(wmutex);
  wait(rmutex);                           writecount++;
  readcount++;                            if (writecount == 1) wait(readTry);
  if (readcount == 1) wait(wrt);          signal(wmutex);
  signal(rmutex);                         wait(wrt);
  signal(readTry);                        /* WRITING */
  /* READING */                           signal(wrt);
  wait(rmutex);                           wait(wmutex);
  readcount--;                            writecount--;
  if (readcount == 0) signal(wrt);        if (writecount == 0) signal(readTry);
  signal(rmutex);                         signal(wmutex);
```

Pseudocode ١١ — Third problem: fair solution (strong FIFO semaphore "order")

```
semaphore mutex = 1, wrt = 1, order = 1;   int readcount = 0;

Reader:                                 Writer:
  wait(order);                            wait(order);
  wait(mutex);                            wait(wrt);
  readcount++;                            signal(order);
  if (readcount == 1) wait(wrt);          /* WRITING */
  signal(mutex);                          signal(wrt);
  signal(order);
  /* READING */
  wait(mutex);
  readcount--;
  if (readcount == 0) signal(wrt);
  signal(mutex);
```

Pseudocode ١٢ — Writer-priority solution as a monitor (Mesa semantics)

```
monitor ReadersWriters {
    int activeReaders = 0, waitingWriters = 0;  boolean writing = false;
    condition okRead, okWrite;

    procedure beginRead() {
        while (writing || waitingWriters > 0) okRead.wait();
        activeReaders++;
    }
    procedure endRead() {
        activeReaders--;
        if (activeReaders == 0) okWrite.signal();
    }
    procedure beginWrite() {
        waitingWriters++;
        while (writing || activeReaders > 0) okWrite.wait();
        waitingWriters--;  writing = true;
    }
    procedure endWrite() {
        writing = false;
        if (waitingWriters > 0) okWrite.signal(); else okRead.broadcast();
    }
}
```

٧,٣ Flowchart (first problem)

The first reader keeps writers out with `wait(wrt)` and the last reader releases the resource with `signal(wrt)`. The variable `readcount` is protected by `mutex`.

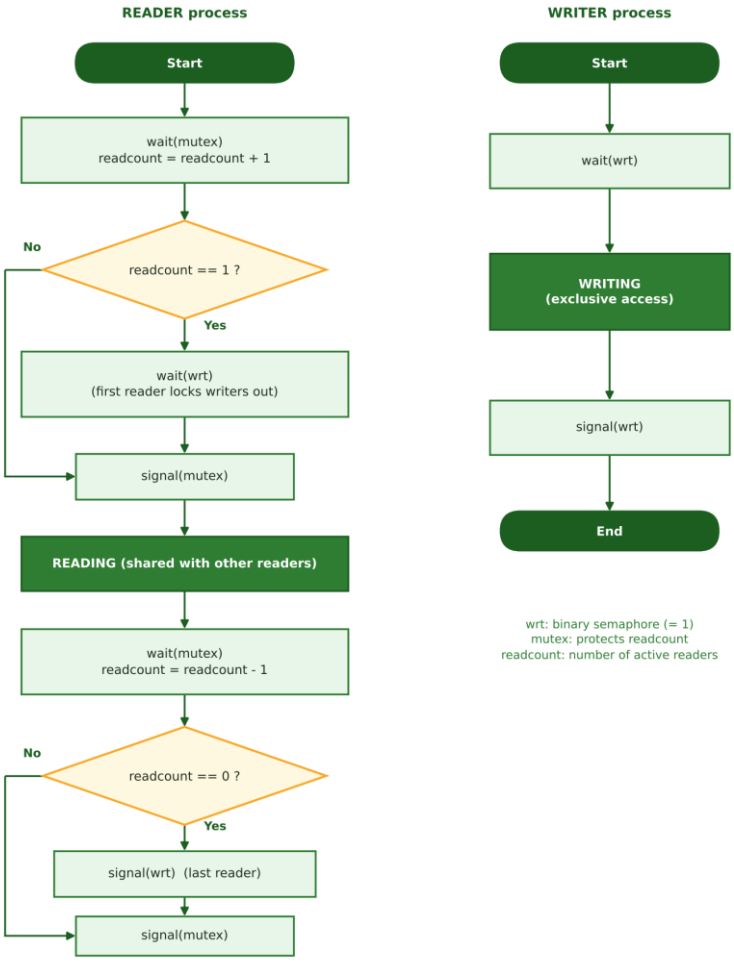


Figure ٧: *Flowchart of the reader and writer processes in the first-problem solution*

٧,٤ Comparison of the three variants

Table ٢: Comparison of the three Readers–Writers variants

Property	Reader priority	Writer priority	Fair (FIFO)
Writer starvation	Possible	No	No
Reader starvation	No	Possible	No
Reader concurrency	Maximal	Reduced	Medium
Bounded waiting	No	No	Yes
Semaphores used	٢ + readcount	٤ + ٢ counters	٣ + readcount
Best suited for	Read-mostly data	Fresh-data critical systems	General purpose

٧,٥ Simulation of the three policies

To measure the difference between the three variants in practice, a discrete-event simulation was written. Assumptions: readers and writers arrive as Poisson processes; the writer arrival rate is ٠,١٥ per time unit while the reader arrival rate varies from ٠,٥ to ٤; read and write durations are exponential with means ١,٠ and ١,٥; the horizon is ٣٠,٠٠٠ time units and results are averaged over three random seeds. Requests still waiting at the end are counted with their current waiting time so that starvation is not hidden. The results are specific to this model; absolute values change with the workload.

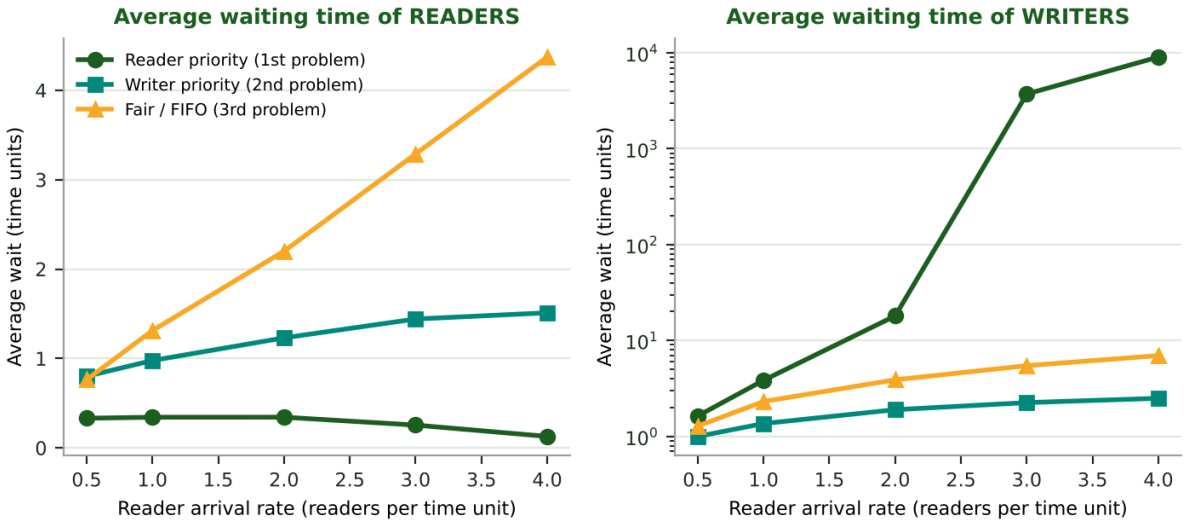


Figure ٨: *Average waiting time of readers and writers (writer axis is logarithmic)*

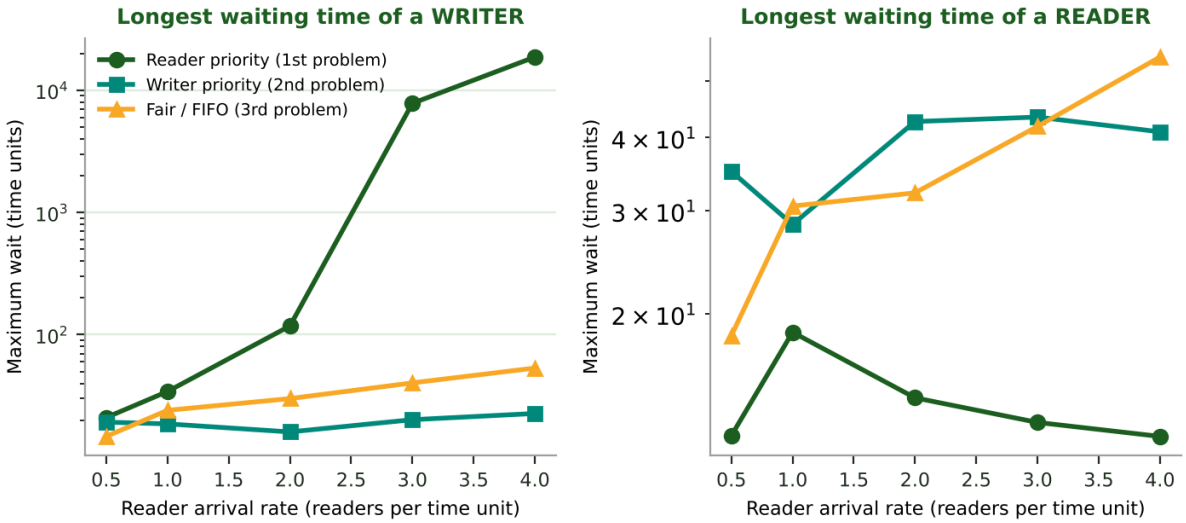


Figure ٩: *Maximum waiting time of a writer and of a reader (logarithmic axes)*

Interpretation: at a reader arrival rate of ٤, the average writer wait under reader priority is about ٩,٠٠٠ time units (effectively starvation, bounded only by the simulation length), versus about ٢,٥ under writer priority and about ٧ under the fair policy. Conversely, a reader waits only about ٠,١٣ units under reader priority, about ١,٥ under writer priority and about ٤,٤ under the fair policy. Fairness is therefore paid for with a higher average reader wait, but it gives a bound on the waiting of both groups.

٧,٦ Analysis

- **Mutual exclusion:** the writer is separated from everyone else by `wrt`; readers deliberately share. **Progress:** holds in all variants. **Bounded waiting:** only in the third variant.
- **Cost:** maintaining `readcount` and at least two semaphores; the second variant needs more semaphores and more complexity. In practice ready-made reader–writer locks (such as `pthread_rwlock` or `ReentrantReadWriteLock`) are used.

٨ Comprehensive Comparison Tables

In this section the five approaches are compared in three tables using all parameters of Section ٢: general characteristics, correctness and safety, and performance and practice.

Table ٣: General characteristics

Property / Parameter	Dekker	Peterson	Semaphore	Monitor	Readers–Writers
Author and year	Th. J. Dekker, c. ١٩٦٢ (Dijkstra, ١٩٦٥)	G. L. Peterson, ١٩٨١	Dijkstra, ١٩٦٥	Hoare / Brinch Hansen, ١٩٧٤	Courtois et al., ١٩٧١
Type and level	Software algorithm	Software algorithm	OS / hardware primitive	Language-level construct	Coordination problem (semaphores or monitors)
Processes supported	٢ (N via generalization)	٢ (N with filter lock)	N	N	N readers + M writers
Hardware / OS support	Atomic read/write + memory fences	Atomic read/write + memory fences	Atomic instruction or OS support	Language / runtime support	Semaphores or monitors
Shared variables (٢ processes)	٣ (flag[٢], turn)	٣ (flag[٢], turn)	١ semaphore (+ kernel queue)	١ implicit lock + conditions	٣ (mutex, wrt, readcount)
Entry + exit statements	٩	٤	٢	٢ (implicit)	٨ reader / ٢ writer
Implementation complexity	High	Low	Low	Medium	Medium–High
Difficulty of proof	Hard	Easy	Medium	Easy (encapsulation)	Medium

Table ٤: Correctness and safety (bounded waiting, progress, …)

Property / Parameter	Dekker	Peterson	Semaphore	Monitor	Readers–Writers
Mutual exclusion	Guaranteed	Guaranteed	Guaranteed (binary semaphore)	Guaranteed and automatic	Writer vs. all; readers share
Progress	Holds	Holds	Holds	Holds	Holds
Bounded waiting	Holds (at most ١ turn)	Holds (at most ١ turn)	Depends on queue (strong: yes, weak: no)	Depends on queue policy	Only the fair variant
Starvation	None	None	Possible (weak semaphore)	Possible (unfair signalling)	Possible (variants ١, ٢)
Deadlock	None (due to back-off)	None	Possible (wrong ordering)	Possible (nested monitors)	Possible if misused
Fairness	Turn-based	Turn-based	FIFO if strong	FIFO with fair queues	Variant dependent
Memory-model issue	Needs memory fences	Needs memory fences	None (handled by OS / library)	None	None
Priority-inversion risk	High (spinning)	High (spinning)	Present	Present	Present

Table ٥: Performance and practice

Property / Parameter	Dekker	Peterson	Semaphore	Monitor	Readers–Writers
Waiting type	Busy waiting	Busy waiting	Blocking (busy in simple version)	Blocking	Blocking
CPU efficiency	Poor	Poor	Good	Good	Good
Scalability	Poor (٢ processes)	Poor (filter O(N))	Excellent	Excellent	Good
Memory growth with N	٢N + ١	٢N - ١	Constant O(١)	Constant O(١)	Constant O(١)
Context-switch overhead	None	None	Yes (when blocking)	Yes (on wait)	Yes
Typical applications	Teaching, historical	Teaching, systems without atomic ops	OS kernels, producer–consumer	Java, C#, pthreads (mutex + cond)	Databases, file systems, caches
Main weakness	Busy wait, intricacy	Busy wait, memory reordering	Error-prone, priority inversion	Language dependent, nested lockout	Starvation in variants ١, ٢

٩ Comparison Charts

٩,١ Qualitative radar chart

Figure ١٠ shows seven criteria on a scale of ١ to ٥, where ٥ is best. The scores are the author's qualitative assessment based on the analysis of the previous sections (for example, “CPU efficiency” is ١ for busy-waiting methods and ٤ for blocking methods) and are not empirical measurements.

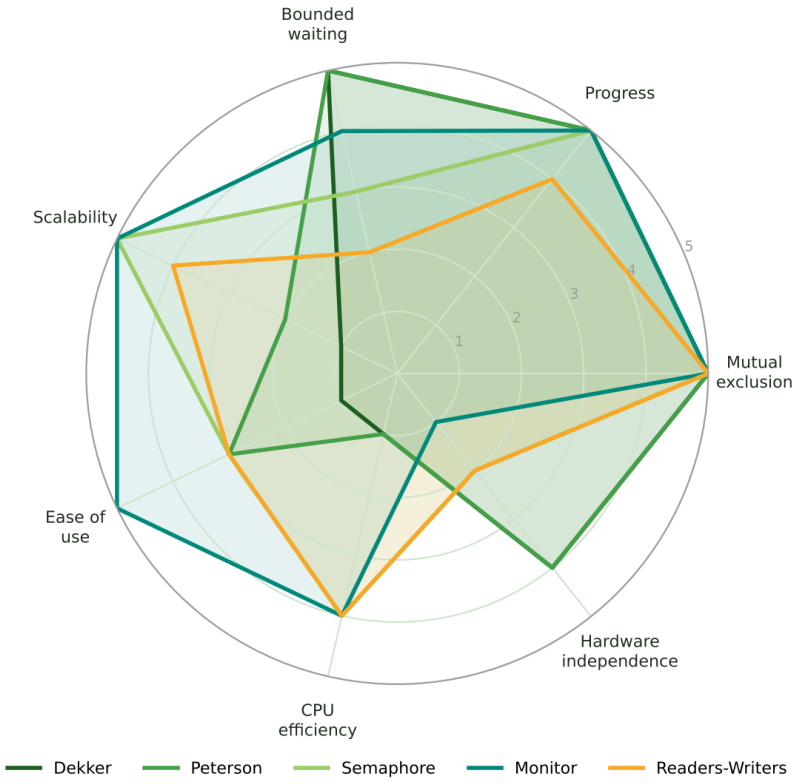


Figure ١٠: *Radar chart for the qualitative comparison of the five approaches*

Table ٦: Qualitative scores (١ to ٥, higher is better)

Criterion	Dekker	Peterson	Semaphore	Monitor	Readers–Writers
Mutual exclusion	٥	٥	٥	٥	٥
Progress	٥	٥	٥	٥	٤
Bounded waiting	٥	٥	٣	٤	٢
Scalability	١	٢	٥	٥	٤
Ease of use	١	٣	٣	٥	٣
CPU efficiency	١	١	٤	٤	٤
Hardware independence	٤	٤	٢	١	٢

٩,٢ Structural metrics

Figure ١١ shows two countable indicators derived from the pseudocode of this article. Semaphores and monitors have the shortest entry/exit protocol; Dekker is the longest with ٩ statements. For the monitor the implicit lock is counted as one variable and condition variables are not counted.

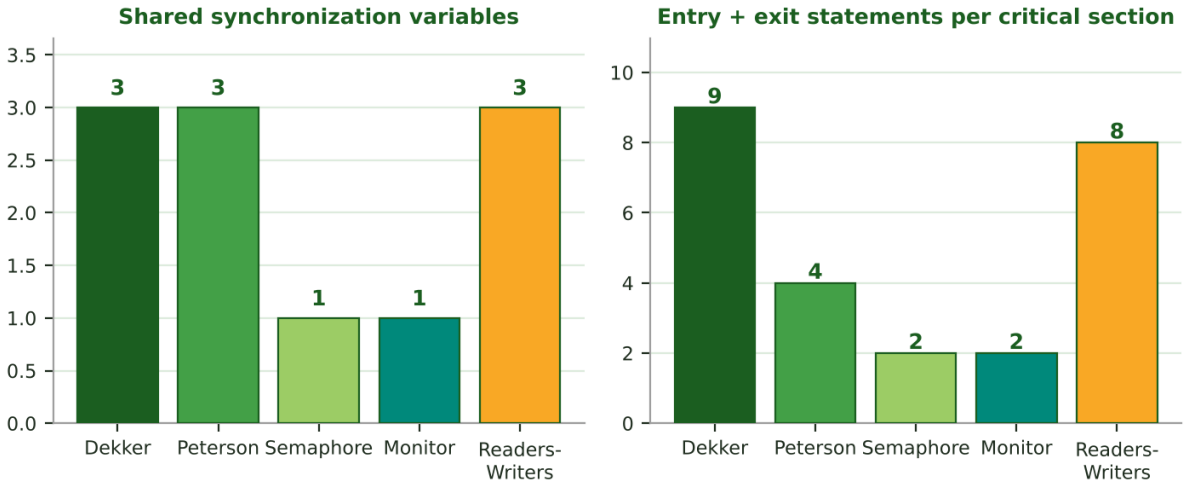


Figure ١١: *Number of shared variables and entry/exit statements*

٩,٣ Memory growth with the number of processes

Figure ١٢ shows that the N-process software methods (Dijkstra's generalization of Dekker with ٢N+١ cells and Peterson's filter lock with ٢N-١ cells) grow shared memory linearly, whereas a semaphore or monitor lock always has one shared variable (the wait queues are kept by the kernel).

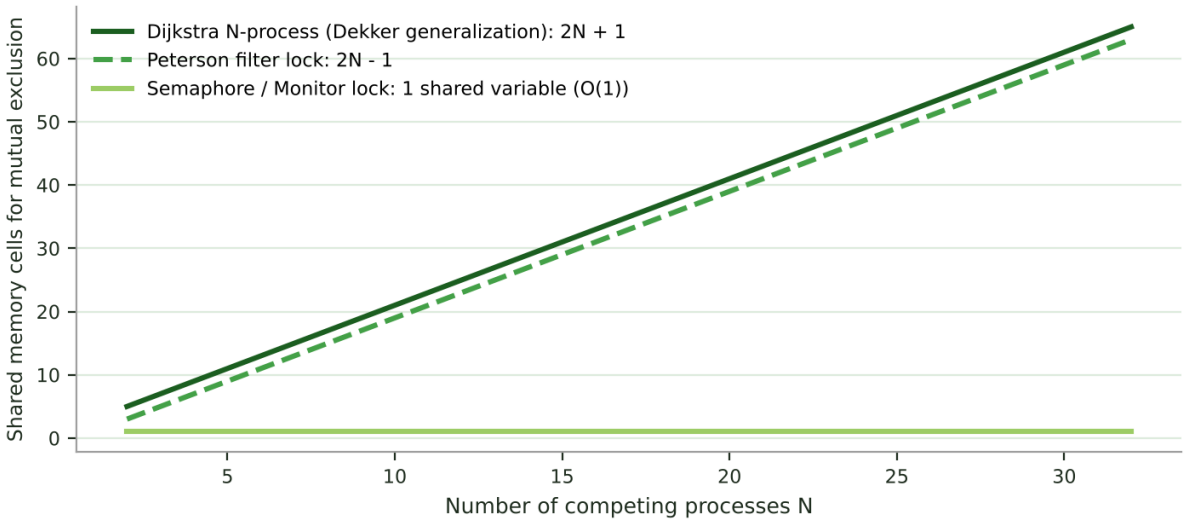


Figure ١٢: *Shared memory cells versus number of processes*

١٠ Algorithm Selection Guide

Figure ١٣ is a simple decision tree for choosing a suitable approach. In practice a modern programmer almost never uses Dekker or Peterson directly; they are mainly valuable for understanding the principles and for environments without atomic-operation support.

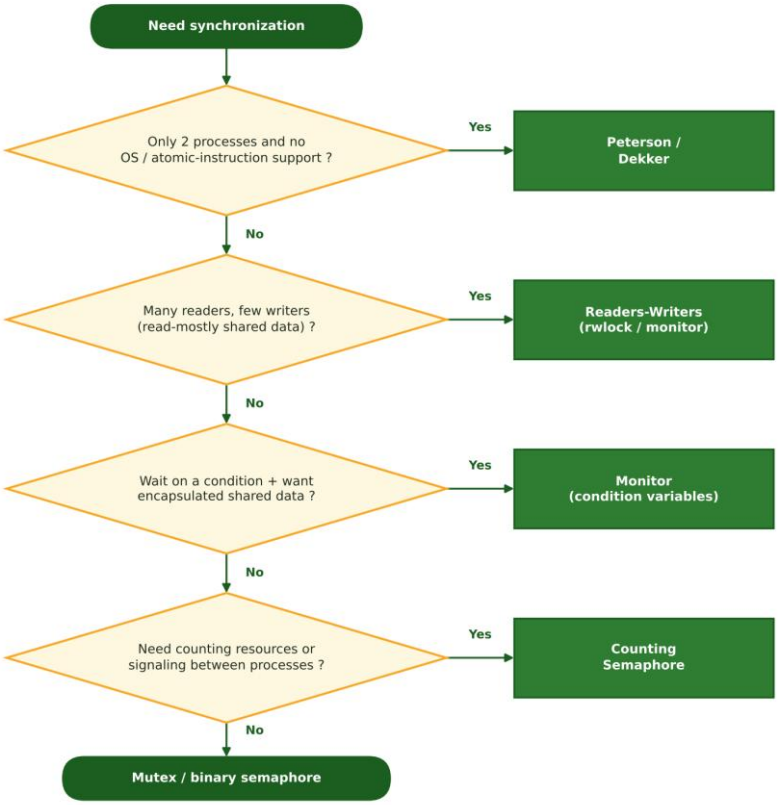


Figure ١٣: *Decision tree for choosing a synchronization method*

Table ٧: Scenario and recommended method

Scenario	Recommended	Reason
OS course, proving correctness	Peterson	Short and provable
Microcontroller without atomic ops	Peterson (with fences)	Needs only reads/writes
Protecting a data structure	mutex / monitor	Simple, safe, blocking
Producer–consumer	Counting semaphore or monitor	Capacity counting and ordering
Read-mostly database	Readers–Writers	Reader concurrency
Starvation must be avoided	Fair FIFO solution	Bounded waiting

١١ Conclusion

This article compared five classic synchronization solutions with pseudocode, flowcharts, charts and comprehensive tables. **Dekker** and **Peterson** show that mutual exclusion is possible with plain reads and writes, but busy waiting, the two-process limit and sensitivity to the memory model restrict their practical use; Peterson is preferable to Dekker because of its simplicity. The **semaphore** is a general and scalable tool, but error-prone. The **monitor**, with encapsulation and automatic mutual exclusion, is the safest and most widely used high-level tool. **Readers–Writers** shows that the choice of priority policy is a direct trade-off between throughput and fairness: the simulation showed that reader priority can starve writers under heavy load, writer priority keeps reader waiting moderate, and the fair policy bounds the waiting of both groups at the cost of a higher average.

Future work: comparison with locks built on hardware instructions (Test-and-Set, Compare-and-Swap), Lamport's Bakery algorithm and lock-free structures, as well as empirical measurements on real hardware.

١٢ References

[١] E. W. Dijkstra, “Cooperating Sequential Processes”, EWD١٢٣, ١٩٦٥; in F. Genuys (ed.), Programming Languages, Academic Press, ١٩٦٨, pp. ٤٣–١١٢.

[٢] G. L. Peterson, “Myths About the Mutual Exclusion Problem”, Information Processing Letters, ١٢(٣), ١٩٨١, pp. ١١٥–١١٦.

[٣] C. A. R. Hoare, “Monitors: An Operating System Structuring Concept”, Communications of the ACM, ١٧(١٠), ١٩٧٤, pp. ٥٤٩–٥٥٧.

[٤] P. J. Courtois, F. Heymans, D. L. Parnas, “Concurrent Control with Readers and Writers”, Communications of the ACM, ١٤(١٠), ١٩٧١, pp. ٦٦٧–٦٦٨.

[٥] P. Brinch Hansen, Operating System Principles, Prentice Hall, ١٩٧٣.

[٦] A. Silberschatz, P. B. Galvin, G. Gagne, Operating System Concepts, Wiley (multiple editions).

[٧] A. S. Tanenbaum, H. Bos, Modern Operating Systems, Pearson (multiple editions).

[٨] M. Herlihy, N. Shavit, The Art of Multiprocessor Programming, Morgan Kaufmann.

[٩] A. B. Downey, The Little Book of Semaphores, Green Tea Press.